RUN: DM Classification and Clustering

Nov 6, 2026 · @shahan

Supervised learning and clustering for Sessional-2, with every recent question solved where it belongs. Allow about 2.5 hours. **Every numerical was recomputed with a script.**

How this run works

Read the idea, try the PYQ in its box, then check the solution right under it. The line to write in the exam is in bold.

Scope: "Supervised Learning and Clustering" maps to syllabus Units 3, 4 and 5 (basic classification, advanced classification and prediction, cluster analysis). This run covers **every topic those three units list**, plus SVM and DBSCAN, which aren't in the syllabus text but were asked in last year's papers. Sir stopped somewhere, so anything past that point is free endsem prep.

What the paper likes: an entropy and information gain table, a Naive Bayes prediction, a k-means run shown round by round, and a single-link dendrogram. All four are pure arithmetic once you know the recipe, and arithmetic can't surprise you.

Videos: StatQuest on YouTube, for Decision Trees, Naive Bayes, Support Vector Machines, K-means and Hierarchical Clustering. Notes are in `Data Mining/Notes/`. The Units 1 to 3 slides are still on Classroom, not downloaded.

Papers used: CSE-501 Sessional-2 2025 and Endsem 2025, the Data Analytics Honours Sessional-2 2025 (the same clustering syllabus), and the CSDS Sessional-2 2026 paper from this year.

Stop 1: Classification basics

Classification learns a model from labelled training data, then predicts the class label of new data. It's **supervised** because the labels are given. Clustering, from Stop 6 on, has no labels, so it's **unsupervised**.

The two steps:

1. **Learning:** build the model from the training set.
2. **Classification:** check accuracy on a separate test set, then use the model on new tuples.

$$\text{Accuracy} = \frac{\text{correctly classified test tuples}}{\text{total test tuples}}$$

Learner	When it works	Examples
Eager	Builds a general model during training	Decision tree, Naive Bayes, SVM
Lazy	Stores the training data, and works only when a query arrives	k-nearest neighbour, case-based reasoning

> **PYQ, Sessional-2 2025 Q3(ii), 1 mark.** Compare the advantages and disadvantages of eager classification versus lazy classification.

Answer:

Basis	Eager	Lazy
Training time	High, the model is built up front	Almost none, data is just stored
Prediction time	Fast	Slow, all the work happens per query
Storage	Only the model	The whole training set
Hypothesis	One global model	Local, built around each query
Adapts to new data	Needs retraining	Just add the tuple

Eager learners pay at training time and predict fast. Lazy learners train instantly and pay at every prediction. It's the difference between studying all semester and starting the night before, except here both actually work.

Stop 2: Decision trees

A decision tree asks one attribute question at each node, splitting the data until each leaf is (nearly) one class. The whole game is picking the **best attribute to split on**: the one that leaves the children purest.

The impurity measures, where p_i is the fraction of class i in a node:

$$\text{Entropy}(D) = -\sum_i p_i \log_2 p_i$$

$$\text{Gini}(D) = 1 - \sum_i p_i^2$$

$$\text{Error}(D) = 1 - \max_i p_i$$

All three are 0 for a pure node. For two classes, entropy peaks at 1, and Gini and error both peak at 0.5, when the split is 50:50.

Information gain is how much a split lowers entropy, weighting each child by its size:

$$\text{Gain}(A) = \text{Entropy}(D) - \sum_v \frac{|D_v|}{|D|} \text{Entropy}(D_v)$$

ID3 picks the highest gain. C4.5 uses **gain ratio**, which divides by the split information to stop many-valued attributes like ID numbers from winning. CART uses Gini.

Handy entropy values to save calculator time: a 1:1 split gives 1, 1:3 gives 0.8113, 2:3 gives 0.9710, 1:4 gives 0.7219, 1:2 gives 0.9183, 2:5 gives 0.8631.

> **PYQ, Sessional-2 2025 Q1, 4 marks.** Ten training examples with attributes A1, A2 and a class (+ or −). (i) What is the entropy of this collection with respect to the positive class? (ii) Calculate the information gains, and which is the best split according to the classification error rate?

Instance	1	2	3	4	5	6	7	8	9	10
A1	T	T	T	F	F	F	F	T	F	F
A2	T	T	F	F	T	T	F	F	T	F
Class	+	+	−	+	−	+	−	+	+	−

Answer: There are 6 positives and 4 negatives. (The A3 column in our copy is scribbled over; if it's numeric, sort its values and test a split point between each pair, the same way.)

(i) Entropy:

$$\text{Entropy}(D) = -\frac{6}{10}\log_2\frac{6}{10} - \frac{4}{10}\log_2\frac{4}{10} = 0.442 + 0.529 = 0.9710$$

(ii) Gains. Split each attribute and count the classes in each branch.

Split	Branch	+	−	Entropy	Error
A1	T (4)	3	1	0.8113	1/4
A1	F (6)	3	3	1.0000	3/6
A2	T (5)	4	1	0.7219	1/5
A2	F (5)	2	3	0.9710	2/5

$$\text{Gain}(A1) = 0.9710 - \left[\tfrac{4}{10}(0.8113) + \tfrac{6}{10}(1.0)\right] = 0.9710 - 0.9245 = 0.0464$$
$$\text{Gain}(A2) = 0.9710 - \left[\tfrac{5}{10}(0.7219) + \tfrac{5}{10}(0.9710)\right] = 0.9710 - 0.8464 = 0.1245$$

Now the classification error. Before any split the error is 1 − 6/10 = 0.4.

$$\text{Error after A1} = \tfrac{4}{10}\cdot\tfrac{1}{4} + \tfrac{6}{10}\cdot\tfrac{3}{6} = 0.1 + 0.3 = 0.4 \quad (\text{gain } 0)$$
$$\text{Error after A2} = \tfrac{5}{10}\cdot\tfrac{1}{5} + \tfrac{5}{10}\cdot\tfrac{2}{5} = 0.1 + 0.2 = 0.3 \quad (\text{gain } 0.1)$$

Entropy = 0.971. Gain(A1) = 0.046, Gain(A2) = 0.125. A2 is the best split by both information gain and classification error (0.3 vs 0.4).

> **PYQ, Endsem 2025 Q4(b), 4.5 marks.** For the 12-day data below, where Play? is the class, describe and calculate the information gain for each attribute, and choose the feature to split at the root of the decision tree.

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No
11	Rainy	Cool	Normal	Strong	No
12	Cloudy	Hot	High	Weak	Yes

Answer: There are 6 Yes and 6 No, so Entropy(D) = 1. Now count each value's Yes and No.

Attribute	Value (count)	Yes	No	Entropy
Weather	Sunny (3)	1	2	0.9183
Weather	Cloudy (4)	4	0	0
Weather	Rainy (5)	1	4	0.7219
Temperature	Hot (5)	3	2	0.9710
Temperature	Mild (5)	3	2	0.9710
Temperature	Cool (2)	0	2	0
Humidity	High (8)	4	4	1
Humidity	Normal (4)	2	2	1
Wind	Weak (5)	4	1	0.7219
Wind	Strong (7)	2	5	0.8631

$$\begin{aligned}
\text{Gain}(\text{Weather}) &= 1 - \left[\tfrac{3}{12}(0.9183) + \tfrac{4}{12}(0) + \tfrac{5}{12}(0.7219)\right] = 1 - 0.5304 = 0.4696 \\
\text{Gain}(\text{Temperature}) &= 1 - \left[\tfrac{5}{12}(0.9710) + \tfrac{5}{12}(0.9710) + \tfrac{2}{12}(0)\right] = 1 - 0.8091 = 0.1909 \\
\text{Gain}(\text{Humidity}) &= 1 - \left[\tfrac{8}{12}(1) + \tfrac{4}{12}(1)\right] = 0 \\
\text{Gain}(\text{Wind}) &= 1 - \left[\tfrac{5}{12}(0.7219) + \tfrac{7}{12}(0.8631)\right] = 1 - 0.8043 = 0.1957
\end{aligned}$$

Weather has the highest gain, 0.470, so it's the root. The Cloudy branch is already pure (all Yes), so it becomes a leaf straight away. Humidity, with gain 0, is useless here: both its branches are exactly 50:50.

Gain ratio (C4.5). Information gain favours attributes with many values: an ID column would split into pure single-row leaves and score a perfect gain while predicting nothing. Gain ratio divides by the split information, which is large for many-valued splits.

$$\text{SplitInfo}(A) = -\sum_{v} \frac{|D_v|}{|D|} \log_2 \frac{|D_v|}{|D|}, \qquad \text{GainRatio}(A) = \frac{\text{Gain}(A)}{\text{SplitInfo}(A)}$$

Gini index (CART). CART uses Gini impurity, and the best split gives the largest drop in impurity.

$$\text{Gini}_A(D) = \sum_{v} \frac{|D_v|}{|D|} \text{Gini}(D_v), \qquad \Delta\text{Gini}(A) = \text{Gini}(D) - \text{Gini}_A(D)$$

> **Practice, same 12-day data.** Find the gain ratio and the Gini reduction for each attribute. Does the root change?

Answer: Gini(D) = 1 − (0.5^2 + 0.5^2) = 0.5.

Attribute	Gain	SplitInfo	Gain ratio	Gini after split	Gini reduction
Weather (3/4/5)	0.4696	1.5546	**0.3021**	0.2444	**0.2556**
Temperature (5/5/2)	0.1909	1.4834	0.1287	0.4000	0.1000
Humidity (8/4)	0	0.9183	0	0.5000	0
Wind (5/7)	0.1957	0.9799	0.1997	0.3714	0.1286

For example, the Weather row works out as:

$$\text{SplitInfo}(\text{Weather}) = -\tfrac{3}{12}\log_2\tfrac{3}{12} - \tfrac{4}{12}\log_2\tfrac{4}{12} - \tfrac{5}{12}\log_2\tfrac{5}{12} = 1.5546$$
$$\text{GainRatio} = \frac{0.4696}{1.5546} = 0.3021$$
$$\text{Gini}_{\text{Weather}} = \tfrac{3}{12}\left(1 - \tfrac{1}{9} - \tfrac{4}{9}\right) + \tfrac{4}{12}(0) + \tfrac{5}{12}\left(1 - \tfrac{1}{25} - \tfrac{16}{25}\right) = 0.2444$$

Weather wins on all three measures, so the root stays Weather. Notice Wind overtakes Temperature under gain ratio, because Temperature's three-way split gets penalised.

Tree pruning. A fully grown tree memorises noise and outliers, which is overfitting. Pruning cuts away branches that don't help on unseen data.

Type	How	Trade-off
Pre-pruning	Stop growing early, when a split's gain falls below a threshold	Fast, but a good split hiding behind a weak one gets missed
Post-pruning	Grow the full tree, then replace subtrees with leaves if the error doesn't rise (cost complexity in CART, pessimistic pruning in C4.5)	More accurate, more computation

Pruned trees are smaller, faster and more accurate on test data.

Stop 3: Naive Bayes

Bayes' theorem flips "how likely is this evidence, given the class" into "how likely is the class, given this evidence". The **naive** part assumes attributes are independent given the class, so the likelihood is just a product.

$$P(C \mid X) = \frac{P(X \mid C)\,P(C)}{P(X)}$$
$$P(X \mid C) = P(x_1 \mid C) \times P(x_2 \mid C) \times \cdots \times P(x_n \mid C)$$

P(X) is the same for every class, so compare only P(X | C) × P(C) and pick the biggest.

The recipe: priors from the class counts, then one conditional per attribute value in X, per class, then multiply.

Zero-probability problem: if one conditional is 0, the whole product becomes 0. The **Laplace correction** adds 1 to every count, and adds the number of values to the denominator.

> **PYQ, Sessional-2 2025 Q2, 4 marks.** Build a Naive Bayes model from the data below, and determine the species of X = (Color = Green, Legs = 2, Height = Tall, Smell = No).

No.	Color	Legs	Height	Smell	Species
1	White	3	Short	Yes	M
2	Green	2	Tall	No	M
3	Green	3	Short	Yes	M
4	White	3	Short	Yes	M
5	Green	2	Short	No	H
6	White	2	Tall	No	H
7	White	2	Tall	No	H
8	White	2	Short	Yes	H

Answer: Priors: P(M) = 4/8 and P(H) = 4/8.

Conditional	M	H
Color = Green	2/4	1/4
Legs = 2	1/4	4/4
Height = Tall	1/4	2/4
Smell = No	1/4	3/4

$$P(X \mid M)\,P(M) = \tfrac{2}{4} \cdot \tfrac{1}{4} \cdot \tfrac{1}{4} \cdot \tfrac{1}{4} \cdot \tfrac{4}{8} = \tfrac{1}{256} = 0.0039$$
$$P(X \mid H)\,P(H) = \tfrac{1}{4} \cdot \tfrac{4}{4} \cdot \tfrac{2}{4} \cdot \tfrac{3}{4} \cdot \tfrac{4}{8} = \tfrac{3}{64} = 0.0469$$

0.0469 > 0.0039, so X is classified as species H.

> **PYQ, Endsem 2025 Q4(a), 4.5 marks.** Using the student data below, use a Naive Bayes classifier to decide whether someone with Excellent attendance, Poor GPA and Lots of effort should be hired.

Name	Attendance	Effort	GPA	Hirable
Sarah	Excellent	Lots	Average	Yes
Dana	Poor	Some	Average	No
Alex	Excellent	Some	Average	No
Annie	Poor	Lots	Poor	Yes
Emily	Excellent	Lots	Excellent	Yes
Pete	Poor	Lots	Excellent	No
John	Poor	Lots	Excellent	No
Kathy	Excellent	Some	Poor	No

Answer: Priors: P(Yes) = 3/8 and P(No) = 5/8.

Conditional	Yes	No
Attendance = Excellent	2/3	2/5
GPA = Poor	1/3	1/5
Effort = Lots	3/3	2/5

$$P(X \mid \text{Yes})\,P(\text{Yes}) = \tfrac{2}{3}\cdot\tfrac{1}{3}\cdot 1\cdot\tfrac{3}{8} = \tfrac{1}{12} = 0.0833$$
$$P(X \mid \text{No})\,P(\text{No}) = \tfrac{2}{5}\cdot\tfrac{1}{5}\cdot\tfrac{2}{5}\cdot\tfrac{5}{8} = \tfrac{1}{50} = 0.02$$

Normalising, P(Yes | X) = 0.0833 / (0.0833 + 0.02) = 0.81.

Hire: Yes, with a posterior of about 81%. For the "discuss" part, note that every hired student put in Lots of effort, so that one attribute does most of the work.

> **PYQ, CSDS Sessional-2 2026 Q2, 6 marks (this year).** Ten students with Study Hours (H) and Attendance (A), class Pass or Fail. (i) Compute the priors. (ii) Compute all the conditional probabilities. (iii) Predict S11 (H = Medium, A = Good) and S12 (H = High, A = Poor).

Student	H	A	Class
S1	High	Good	Pass
S2	Medium	Average	Pass
S3	Low	Good	Fail
S4	Medium	Poor	Pass
S5	Low	Average	Fail
S6	Medium	Poor	Fail
S7	High	Average	Pass
S8	Low	Poor	Fail
S9	High	Good	Pass
S10	Medium	Average	Pass

Answer:

(i) P(Pass) = 6/10 and P(Fail) = 4/10.

(ii)

Conditional	Pass (6)	Fail (4)
H = High	3/6	0/4
H = Medium	3/6	1/4
H = Low	0/6	3/4
A = Good	2/6	1/4
A = Average	3/6	1/4
A = Poor	1/6	2/4

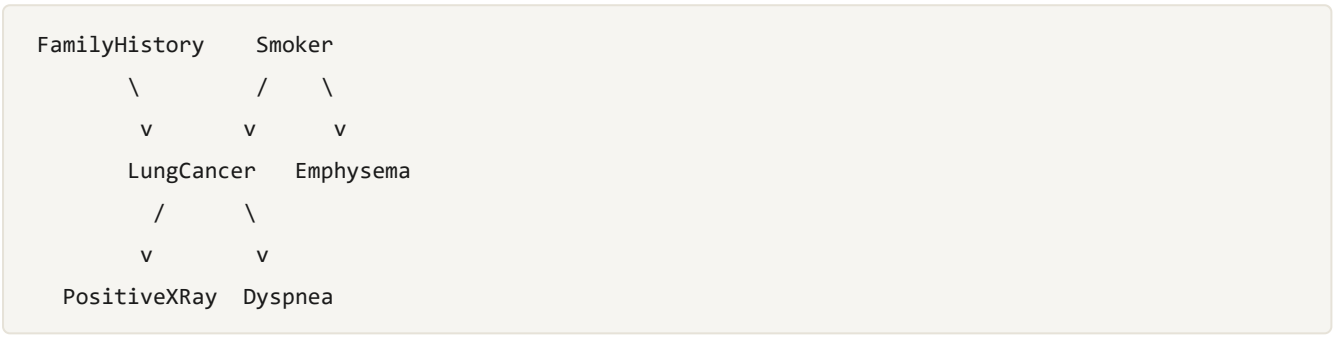
(iii)

$$\text{S11: } P(\text{Pass}) \propto \tfrac{3}{6}\cdot\tfrac{2}{6}\cdot\tfrac{6}{10} = 0.100, \qquad P(\text{Fail}) \propto \tfrac{1}{4}\cdot\tfrac{1}{4}\cdot\tfrac{4}{10} = 0.025$$
$$\text{S12: } P(\text{Pass}) \propto \tfrac{3}{6}\cdot\tfrac{1}{6}\cdot\tfrac{6}{10} = 0.050, \qquad P(\text{Fail}) \propto \tfrac{0}{4}\cdot\tfrac{2}{4}\cdot\tfrac{4}{10} = 0$$

S11 = Pass, and S12 = Pass. S12's Fail score is 0 only because no Fail student had High hours. Mention that the Laplace correction would avoid the hard zero; the answer stays Pass either way.

Bayesian belief networks (Unit 4) drop the naive assumption. Real attributes are often dependent: smoking affects both lung cancer and bronchitis. A BBN models this with two parts:

1. A **directed acyclic graph**: nodes are variables, and an arrow from X to Y means Y depends directly on X. Each node is conditionally independent of its non-descendants, given its parents.
2. A **conditional probability table (CPT)** at each node, giving P(node | parents).



The joint probability factorises along the graph:

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P\left(x_i \mid \text{Parents}(x_i)\right)$$

Naive Bayes is the special case where the class is the only parent of every attribute. A BBN allows any dependency, shown as a DAG with a CPT at each node.

Stop 4: Support Vector Machines

An SVM draws the boundary that keeps the **widest possible gap** between the two classes. Many lines separate the data, and the SVM picks the one with the biggest safety margin, so it generalises best.



$$\text{Hyperplane: } \mathbf{w}\cdot\mathbf{x} + b = 0$$
$$\text{Margin: } \frac{2}{\|\mathbf{w}\|}$$
$$\text{Goal: } \min \tfrac{1}{2}\|\mathbf{w}\|^2 \text{ subject to } y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1$$

- **Support vectors** are the training points closest to the boundary. Only they decide the hyperplane; delete any other point and nothing changes.
- **Soft margin** allows a few points on the wrong side, with a penalty C, for noisy data.

- **Kernel trick:** when the data isn't linearly separable, map it to a higher dimension where it is, using a kernel such as polynomial, RBF (Gaussian) or sigmoid, without computing the mapping explicitly.

> **PYQ, Endsem 2025 Q4(c), 4.5 marks.** Explain linear SVM. Discuss the advantages and disadvantages of SVM.

Answer: A linear SVM separates two classes with the hyperplane $w \cdot x + b = 0$ that maximises the margin $2 / \|w\|$ between the nearest points of each class, the support vectors. A new point is classified by the sign of $w \cdot x + b$.

Advantages	Disadvantages
High accuracy, and resists overfitting thanks to the maximum margin	Slow to train on large datasets
Works in high dimensions, even when features outnumber samples	Choosing the kernel and parameters (C, gamma) is tricky
Memory-efficient: only the support vectors are kept	No direct probability output
Handles non-linear data through kernels	Hard to interpret compared with a decision tree

Linear SVM = the maximum-margin hyperplane, set by the support vectors. Great accuracy, slow on big data.

> **PYQ, Sessional-2 2025 Q3(i), 3 marks.** SVMs are highly accurate, but slow to train on a large set of data tuples. Discuss how to overcome this difficulty, and develop a scalable SVM algorithm for efficient classification of large datasets.

Answer: Training cost grows roughly with the square of the number of tuples, but only the tuples near the boundary matter. So shrink the data first, keeping the detail only where the boundary is. This is **CB-SVM** (clustering-based SVM), from Han and Kamber.

1. Build a **micro-cluster tree** (a CF-tree, as in BIRCH) separately for the positive and the negative tuples, in one scan of the data. Each node stores just a cluster summary: count, linear sum and squared sum.
2. Train an SVM on the **centroids** of the top-level clusters only. That's a tiny training set.
3. Find the clusters that lie **near the boundary**, within the margin or close to it. **Decluster** them, replacing each with its child clusters.
4. Retrain on the new set of centroids, and repeat until reaching the leaves.

Far-away clusters never get expanded, because they can't be support vectors.

Other options worth a line each: sampling the data, chunking (train on subsets, keep their support vectors), and SMO, which breaks the optimisation into the smallest possible pieces.

Summarise the data with hierarchical micro-clusters, train on centroids, and expand only the clusters near the boundary.

Stop 5: Linear regression and MSE

This appeared in this year's CSDS paper, so it's cheap insurance. Regression predicts a **number**, not a class. Simple linear regression fits a straight line through the points that minimises the squared vertical errors.

$$\hat{y} = wx + b$$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$w = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}, \qquad b = \bar{y} - w\bar{x}$$

The formulas for w and b come from setting the derivatives of the MSE with respect to w and b to zero. Squaring makes every error positive and punishes big misses more than small ones.

> **PYQ, CSDS Sessional-2 2026 Q1(iii) (this year).** Find the Mean Squared Error for this data.

X	2	3	5	6	8
Y-true	6	4	5	8	10
Y-pred	7	5	6	7	9

Answer: The errors y − ŷ are −1, −1, −1, +1, +1.

$$\text{MSE} = \frac{(-1)^2 + (-1)^2 + (-1)^2 + 1^2 + 1^2}{5} = \frac{5}{5} = 1$$

MSE = 1.

Non-linear regression. When the points curve, transform the problem back into a linear one. A polynomial such as $y = w_0 + w_1x + w_2x^2 + w_3x^3$ becomes linear by setting $x_1 = x$, $x_2 = x^2$ and $x_3 = x^3$, then solving by least squares as usual. Exponential models become linear after taking logs. Regression trees and model trees handle the rest.

Predictor error measures, for numeric predictions, where d is the number of test tuples:

$$\text{MAE} = \frac{1}{d}\sum|y_i - \hat{y}_i|, \qquad \text{MSE} = \frac{1}{d}\sum(y_i - \hat{y}_i)^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}$$

$$\text{Relative absolute error} = \frac{\sum|y_i - \hat{y}_i|}{\sum|y_i - \bar{y}|}, \qquad \text{Relative squared error} = \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

For the CSDS data above, MAE = 5/5 = 1 and RMSE = 1 too, because every error is exactly ±1.

Stop 5B: Evaluating classifiers

The confusion matrix, for a two-class problem with positive as the class of interest:

	Predicted yes	Predicted no
Actual yes	TP	FN
Actual no	FP	TN

$$\text{Accuracy} = \frac{TP + TN}{P + N}, \qquad \text{Error rate} = 1 - \text{Accuracy}$$

$$\text{Sensitivity (recall)} = \frac{TP}{P} = \frac{TP}{TP + FN}, \qquad \text{Specificity} = \frac{TN}{N} = \frac{TN}{TN + FP}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Accuracy lies on imbalanced data. If 97% of patients are healthy, a model that always says "healthy" scores 97% accuracy and 0% sensitivity. That's why sensitivity and specificity exist.

> **Practice.** A cancer test is run on 1000 people, 100 of whom have cancer. It flags 120 people, 90 of whom really have cancer. Find the accuracy, sensitivity, specificity and precision.

Answer: TP = 90 and FN = 100 − 90 = 10. FP = 120 − 90 = 30, and TN = 900 − 30 = 870.

$$\text{Accuracy} = \frac{90 + 870}{1000} = 0.96$$
$$\text{Sensitivity} = \frac{90}{100} = 0.90$$
$$\text{Specificity} = \frac{870}{900} = 0.967$$
$$\text{Precision} = \frac{90}{120} = 0.75$$

Accuracy 96%, sensitivity 90%, specificity 96.7%, precision 75%. One in four positive results is a false alarm.

Accuracy evaluation methods, which decide how the data gets split for testing:

Method	How	Note
Holdout	Randomly split, usually 2/3 for training and 1/3 for testing	Simple, but the estimate depends on one lucky or unlucky split
Random subsampling	Repeat holdout k times, and average the accuracies	Smooths out the luck
k-fold cross-validation	Split into k equal folds. Each fold is the test set once while the other k − 1 train. Average the k results	Every tuple is tested exactly once. k = 10 is standard. Leave-one-out uses k = n
Bootstrap	Sample n tuples **with replacement** for training, and test on the tuples never picked	About 63.2% of the tuples get picked, since $1 - (1 - 1/n)^n \to 1 - e^{-1}$

The .632 bootstrap combines both sets:

$$\text{Acc}(M) = \frac{1}{k} \sum_{i=1}^{k} (0.632 \times \text{Acc}_{\text{test}_i} + 0.368 \times \text{Acc}_{\text{train}_i})$$

Stop 5C: Ensembles and k-nearest neighbour

An **ensemble** combines many classifiers, so one model's mistakes get outvoted by the others.

	Bagging	Boosting (AdaBoost)
Training sets	k bootstrap samples, drawn independently	Sequential: each round reweights the tuples
Focus	Every tuple equally	Tuples the previous models got wrong get higher weights
Combining	Equal majority vote	Weighted vote: more accurate models count more
Effect	Reduces variance, robust to noise	Reduces bias, usually more accurate, but can overfit noise
Example	Random forest is bagging plus random feature choice	AdaBoost

In AdaBoost, a model's vote weight is log((1 − error) / error), so a model with 10% error gets a much bigger say than one with 40%.

k-nearest neighbour is the classic lazy learner. Store all the training tuples. For a new tuple, find the k closest by distance, and take the majority class (or the mean, for prediction). Normalise the attributes first, so a salary in lakhs doesn't drown out an age in years. Choose an odd k to avoid ties.

> **Practice.** Training data: (7, 7) Bad, (7, 4) Bad, (3, 4) Good, (1, 4) Good. Classify (3, 7) with k = 3, using Euclidean distance.

Answer: Squared distances from (3, 7):

Point	Class	Squared distance
(7, 7)	Bad	16 + 0 = 16
(7, 4)	Bad	16 + 9 = 25
(3, 4)	Good	0 + 9 = 9
(1, 4)	Good	4 + 9 = 13

The 3 nearest are (3, 4) Good, (1, 4) Good and (7, 7) Bad.

Majority 2 to 1, so (3, 7) is classified Good.

Stop 6: k-means

Clustering groups similar objects with **no labels given**. Good clusters have high similarity inside and low similarity between them.

Features required of a clustering algorithm: scalability to large data, the ability to handle different attribute types, clusters of arbitrary shape, minimal domain knowledge needed for parameters, robustness to noise and outliers, insensitivity to the order of input, handling high dimensions, constraints, and interpretable results.

Categories of clustering methods:

Category	Idea	Examples
Partitioning	Split into k groups, then improve by relocation	k-means, k-medoids (PAM, CLARA)
Hierarchical	Build a tree by merging or splitting	AGNES, DIANA, BIRCH
Density-based	Grow clusters through dense regions	DBSCAN, OPTICS
Grid-based	Quantise the space into cells	STING, CLIQUE
Model-based	Fit a statistical model to each cluster	EM, COBWEB

Data types and dissimilarity measures, where d(i, j) is the dissimilarity between objects i and j:

Data type	Dissimilarity
Interval-scaled (height, weight)	Standardise first, then Euclidean, Manhattan or Minkowski
Symmetric binary (gender)	(r + s) / (q + r + s + t)
Asymmetric binary (test positive)	(r + s) / (q + r + s), ignoring the 0-0 matches t (the Jaccard distance)
Nominal (colour)	(p − m) / p, with m matches out of p attributes
Ordinal (bronze, silver, gold)	Replace by rank, map to [0, 1] with (r − 1) / (M − 1), then treat as interval
Ratio-scaled (growth)	Take logs, then treat as interval
Mixed	Weighted average of the per-attribute dissimilarities

For binary data, q counts the attributes that are 1 for both objects, r those that are 1 for i only, s those that are 1 for j only, and t those that are 0 for both.

$$d_{\text{Minkowski}}(i,j) = \left(\sum_f |x_{if} - x_{jf}|^p \right)^{1/p} \qquad (p = 1 \text{ Manhattan},\ p = 2 \text{ Euclidean})$$

> **Practice (the Han and Kamber patient example).** Six asymmetric binary attributes (fever, cough, test1 to test4), with yes and positive as 1. Jack = 1 0 1 0 0 0, Mary = 1 0 1 0 1 0, Jim = 1 1 0 0 0 0. Which pair is most alike?

Answer:

$$d(\text{Jack}, \text{Mary}) = \frac{0+1}{2+0+1} = 0.33$$
$$d(\text{Jack}, \text{Jim}) = \frac{1+1}{1+1+1} = 0.67$$
$$d(\text{Jim}, \text{Mary}) = \frac{1+2}{1+1+2} = 0.75$$

Jack and Mary are the most alike (0.33). Jim and Mary are the least alike (0.75).

k-means, the partitioning workhorse:

1. Pick k initial centroids (the question usually gives them).
2. Assign every point to its nearest centroid.
3. Recompute each centroid as the mean of its points.
4. Repeat steps 2 and 3 until no point changes cluster.

$$\text{Euclidean: } d(p,q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$
$$\text{Manhattan: } d(p,q) = |x_1 - x_2| + |y_1 - y_2|$$

Its weak spots: you must choose k, it's sensitive to the starting centroids and to outliers (the mean gets dragged), and it only finds round, convex clusters.

> **PYQ, Endsem 2025 Q5(a), 4.5 marks.** Cluster A1(2, 10), A2(2, 5), A3(8, 4), B1(5, 8), B2(7, 5), B3(6, 4), C1(1, 2), C2(4, 9) into three clusters with Euclidean distance. The initial centres are A1, B1 and C1. Use k-means, showing every round until the solution is reached.

Answer:

Round 1, with centres c1 = (2, 10), c2 = (5, 8), c3 = (1, 2):

Point	to c1	to c2	to c3	Cluster
A1	0	3.61	8.06	1
A2	5.00	4.24	3.16	3
A3	8.49	5.00	7.28	2
B1	3.61	0	7.21	2
B2	7.07	3.61	6.71	2
B3	7.21	4.12	5.39	2
C1	8.06	7.21	0	3
C2	2.24	1.41	7.62	2

The clusters are {A1}, {A3, B1, B2, B3, C2} and {A2, C1}. The new centres are the means:

$$c_1 = (2, 10), \quad c_2 = \left(\tfrac{8+5+7+6+4}{5}, \tfrac{4+8+5+4+9}{5}\right) = (6, 6), \quad c_3 = (1.5, 3.5)$$

Round 2, with centres (2, 10), (6, 6), (1.5, 3.5): C2 is now 2.24 from c1 against 3.61 from c2, so it moves to cluster 1. Everything else stays.

The clusters are {A1, C2}, {A3, B1, B2, B3} and {A2, C1}, with new centres (3, 9.5), (6.5, 5.25) and (1.5, 3.5).

Round 3: B1 is now 2.50 from c1 against 3.13 from c2, so it moves to cluster 1.

The clusters are {A1, B1, C2}, {A3, B2, B3} and {A2, C1}, with new centres (3.67, 9), (7, 4.33) and (1.5, 3.5).

Round 4: every point keeps its cluster, so the algorithm stops.

Final clusters: {A1, B1, C2}, {A3, B2, B3}, {A2, C1}, with centroids (3.67, 9), (7, 4.33) and (1.5, 3.5). It converges in 4 rounds.

> **PYQ, Data Analytics Sessional-2 2025 Q4, 5 marks.** Points P1 to P12: (185, 72), (170, 56), (168, 60), (179, 68), (182, 72), (188, 77), (180, 71), (180, 70), (183, 84), (180, 88), (180, 67), (177, 76). Starting centroids (185, 72) and (170, 56). Use k-means with Manhattan distance and show the clusters after 2 rounds.

Answer:

Round 1, with c1 = (185, 72) and c2 = (170, 56):

Point	to c1	to c2	Cluster
P1 (185, 72)	0	31	1
P2 (170, 56)	31	0	2
P3 (168, 60)	29	6	2
P4 (179, 68)	10	21	1
P5 (182, 72)	3	28	1
P6 (188, 77)	8	39	1
P7 (180, 71)	6	25	1
P8 (180, 70)	7	24	1
P9 (183, 84)	14	41	1
P10 (180, 88)	21	42	1
P11 (180, 67)	10	21	1
P12 (177, 76)	12	27	1

$$c_1 = \left(\tfrac{1814}{10}, \tfrac{745}{10}\right) = (181.4, 74.5)$$
$$c_2 = \left(\tfrac{170+168}{2}, \tfrac{56+60}{2}\right) = (169, 58)$$

Round 2: every point is still nearer its old centroid. For example, P4 is 8.9 from c1 against 20 from c2, and P3 is 27.9 against 3. No point moves.

After 2 rounds: Cluster 1 = {P1, P4, P5, P6, P7, P8, P9, P10, P11, P12}, centroid (181.4, 74.5). Cluster 2 = {P2, P3}, centroid (169, 58).

Stop 7: k-medoids (PAM)

k-medoids uses an **actual data point**, the medoid, as each cluster's centre instead of a mean. One outlier can drag a mean far away, but it can't move a medoid much, so k-medoids is **robust to outliers**. The price is speed.

PAM (Partitioning Around Medoids):

1. Pick k points as the initial medoids.
2. Assign every point to its nearest medoid, and add up the total cost (the sum of distances).
3. Try swapping a medoid with a non-medoid. Keep the swap only if the total cost drops.
4. Repeat until no swap helps.

> **PYQ, Data Analytics Sessional-2 2025 Q2, 3 marks.** Points P1 = (2, 6, 4), P2 = (3, 4, 7), P3 = (8, 1, 2), P4 = (7, 2, 3), P5 = (6, 5, 8), P6 = (4, 7, 6). Picking the initial medoids P1, P3 and P5, show the final clusters formed.

Answer: Use Manhattan distance, the usual choice for PAM, and say so.

Point	to P1	to P3	to P5	Nearest
P1	0	13	9	P1
P2	6	13	5	P5
P3	13	0	12	P3
P4	10	3	9	P3
P5	9	12	0	P5
P6	5	14	6	P1

$$\text{Total cost} = 0 + 5 + 0 + 3 + 0 + 5 = 13$$

Now test swaps. For example, swapping P5 for P2 gives exactly the same groups, at the same cost of 13, and swapping P3 for P4 also costs 13. No swap brings the cost below 13 (a brute-force check of every medoid triple confirms 13 is the minimum), so PAM stops.

Final clusters: {P1, P6}, {P3, P4}, {P2, P5}, with total cost 13. Euclidean distance gives the same three clusters, so either assumption gets full marks.

Stop 8: Hierarchical clustering

Agglomerative (bottom-up, AGNES): start with every point as its own cluster, and repeatedly merge the two closest clusters until one is left. **Divisive** (top-down, DIANA) does the reverse. The result is a **dendrogram**, a tree whose merge heights are the merge distances. You don't need to choose k in advance: cut the tree at any height.

Linkage	Distance between clusters	Tends to give
Single	The closest pair of points (min)	Long, chained clusters
Complete	The farthest pair (max)	Compact clusters
Average	The mean of all pairs	Somewhere in between

The single-link recipe: merge the smallest entry in the matrix, then update the new cluster's distance to each other cluster as the **minimum** of the old two. Repeat.

Divisive (DIANA) runs the other way. Start with every object in one cluster. Split the cluster with the largest diameter: the object with the highest average dissimilarity to the others starts a splinter group, and objects closer to the splinter group than to the rest move across. Repeat until every object is alone. It's rarer in practice, because there are $2^{n-1} - 1$ ways to split n objects in two, so it relies on heuristics.

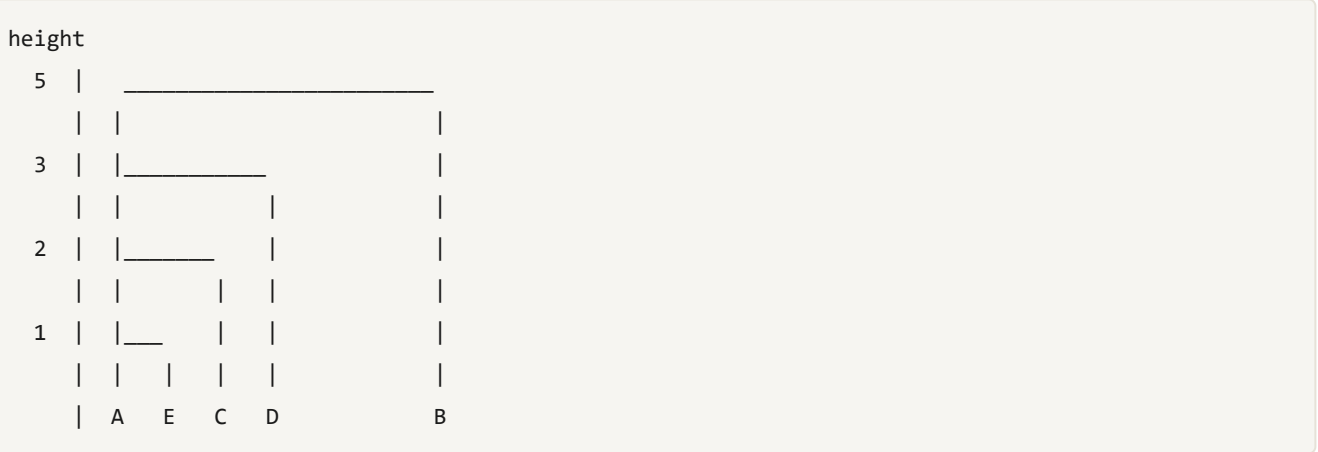
Merges and splits can't be undone, in both directions. That's the main weakness of hierarchical clustering. A bad early merge stays forever.

> **PYQ, Endsem 2025 Q5(c), 4.5 marks.** Explain any one algorithm of hierarchical clustering. For objects A to E with the distance matrix below, what dendrogram does single-link hierarchical clustering return?

	A	B	C	D	E
A	0	9	2	3	1
B		0	8	6	5
C			0	7	10
D				0	4
E					0

Answer: Explain agglomerative single-link using the recipe above, then run it.

Step	Smallest distance	Merge	Updated distances
1	A–E = 1	{A, E}	to B: min(9, 5) = 5; to C: min(2, 10) = 2; to D: min(3, 4) = 3
2	AE–C = 2	{A, E, C}	to B: min(5, 8) = 5; to D: min(3, 7) = 3
3	AEC–D = 3	{A, E, C, D}	to B: min(5, 6) = 5
4	AECD–B = 5	{A, B, C, D, E}	done



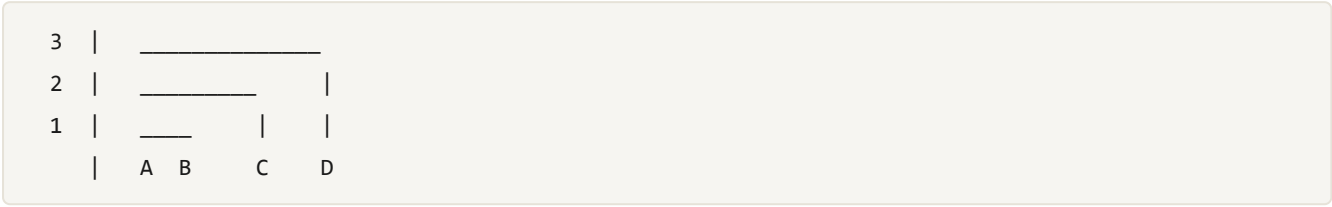
The dendrogram merges A and E at 1, adds C at 2, D at 3, and B at 5. B joins last, so it's the natural outlier.

> **PYQ, Data Analytics Sessional-2 2025 Q3, 3 marks.** Use single-link agglomerative clustering on the matrix below and show the dendrogram. Can a dendrogram tell you the best number of clusters?

	A	B	C	D
A	0	1	4	5
B		0	2	6
C			0	3
D				0

Answer:

Step	Merge	At distance	Updated
1	{A, B}	1	to C: min(4, 2) = 2; to D: min(5, 6) = 5
2	{A, B, C}	2	to D: min(5, 3) = 3
3	{A, B, C, D}	3	done



Merges: A + B at 1, then C at 2, then D at 3. Yes, a dendrogram suggests the number of clusters: cut across the **longest vertical gap**, where no merges happen for the biggest stretch of distance, and count the branches you cross. Here the gaps are all equal (1 each), so this data shows no strong natural grouping, which is itself a fair answer.

Stop 9: DBSCAN, outliers and high dimensions

DBSCAN (density-based clustering) grows clusters through dense regions. It needs two parameters: **ε** (the neighbourhood radius) and **MinPts**.

Point type	Rule
Core	Has at least MinPts points within ε, itself included
Border	Within ε of a core point, but not core itself
Noise	Neither: no core point reaches it

A cluster is a maximal set of **density-connected** points: start from any core point, add everything within ε, and keep expanding through the core points you reach. It finds the number of clusters by itself.

> **PYQ, Endsem 2025 Q5(b)(i); Data Analytics Sessional-2 2025 Q1, 4 marks.** Why can DBSCAN (and OPTICS) find clusters of arbitrary shape, when partition-based algorithms fail to?

Answer: k-means assigns every point to its nearest centroid, so the boundary between two clusters is always the straight line halfway between their centres. That makes every cluster a convex, roughly spherical blob around its centre. A ring or a crescent gets chopped into pieces.

DBSCAN has no centre at all. It follows **density**, chaining from core point to core point, so a cluster can snake along any path the dense region takes, however curved. OPTICS orders points by reachability distance, so it handles clusters of varying density too.

Partitioning measures distance to a centre, which forces convex shapes. Density-based methods follow connected dense regions, whatever their shape.

> **PYQ, Data Analytics Sessional-2 2025 Q4(a).** How can DBSCAN be used to detect outliers?

Answer: DBSCAN labels every point that isn't core and isn't reachable from a core point as **noise**. Those noise points are exactly the outliers: they sit in sparse regions that no dense cluster reaches. Unlike k-means, outliers can't distort the clusters, because they never join one.

DBSCAN's noise points are the outliers, found for free as a by-product of clustering.

> **PYQ, Endsem 2025 Q5(b)(ii).** How does the curse of dimensionality affect distance-based clustering algorithms?

Answer: As the number of dimensions grows:

- **Distances concentrate.** The nearest and the farthest neighbour end up almost equally far away, so "closest" loses its meaning.
- **Data becomes sparse.** The volume grows exponentially, so any fixed number of points thins out, and density-based notions like ε-neighbourhoods stop working.
- **Irrelevant features add noise** to every distance, hiding the real structure.

The fixes are dimensionality reduction (PCA, feature selection) or subspace clustering such as CLIQUE.

In high dimensions all points look roughly equally far apart, so distance-based clustering loses its ability to tell similar from dissimilar.

Stop 10: Web mining and text mining

The last line of Unit 5, and usually worth a short theory answer.

Web mining discovers patterns from the web, and splits three ways:

Type	Mines	Example
Web content mining	The text, images and data on pages	Classifying pages by topic, extracting product prices
Web structure mining	The hyperlink graph	PageRank, finding hubs and authorities (HITS)
Web usage mining	Server logs and clickstreams	Recommending pages, improving site layout

Text mining finds patterns in unstructured text. The pipeline: tokenise, remove stop words, stem ("mining" and "mined" become "mine"), then represent each document as a term vector, usually weighted by **TF-IDF**, so that words frequent in one document but rare overall score highest.

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \times \log \frac{N}{\text{df}(t)}$$

Then apply the usual tools: classification (spam filtering), clustering (grouping news stories), and association (words that co-occur). Documents are compared with cosine similarity.

Web mining is content, structure or usage. Text mining turns documents into TF-IDF vectors, then classifies, clusters or associates them.

That's the run. Refill the tea, and give the cat a pat for being a perfectly clustered point of one.